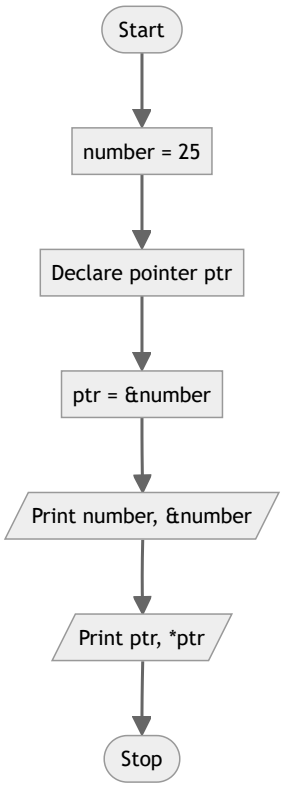
Exp 1 · Pointers · Flowchart



Exp 1 · Pointers · Program

```
#include <iostream>
using namespace std;

int main()
{
    int number = 25;
    int *ptr;

    ptr = &number;

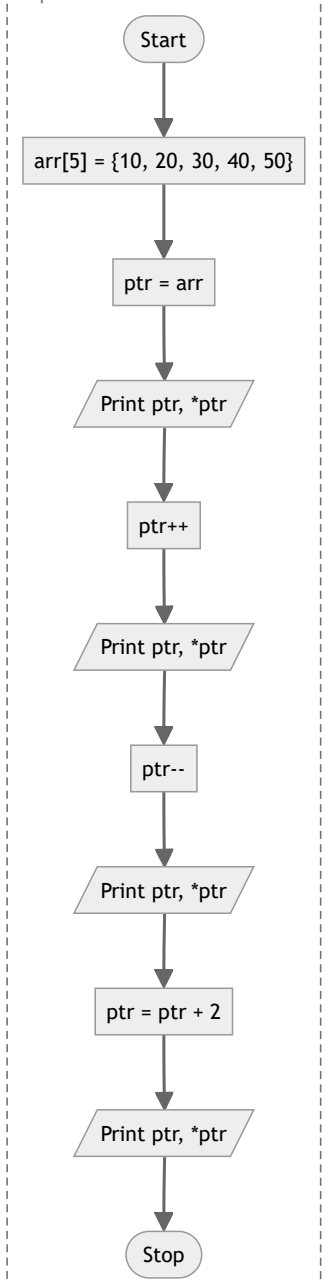
    cout << "Value of number = " << number << endl;
    cout << "Address of number = " << &number << endl;
    cout << "Pointer stores = " << ptr << endl;
    cout << "Value using pointer = " << *ptr << endl;

    return 0;
}
```

Exp 1 · Pointers · Output

```
Value of number = 25
Address of number = 0xbb8cdff8c4
Pointer stores = 0xbb8cdff8c4
Value using pointer = 25
```

Exp 2 · Pointer arithmetic · Flowchart



Exp 2 · Pointer arithmetic · Program

```
#include <iostream>
using namespace std;

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr;

    cout << "Initial Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr++;
    cout << "\nAfter Increment" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr--;
    cout << "\nAfter Decrement" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr = ptr + 2;
    cout << "\nAfter Adding 2" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    return 0;
}
```

Exp 2 · Pointer arithmetic · Output

```
Initial Address : 0x45c67ffa90
Value : 10

After Increment
Address : 0x45c67ffa94
Value : 20

After Decrement
Address : 0x45c67ffa90
Value : 10

After Adding 2
Address : 0x45c67ffa98
Value : 30
```

Exp 3 · Array insertion · Output (1/3)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 1
Enter value: 5

Updated Array:
5 10 20 30 40 50
```

Exp 3 · Array insertion · Output (2/3)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 2
Enter Position: 3
Enter Value: 25

Updated Array:
10 20 25 30 40 50
```

Exp 3 · Array insertion · Output (3/3)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 3
Enter Value: 60

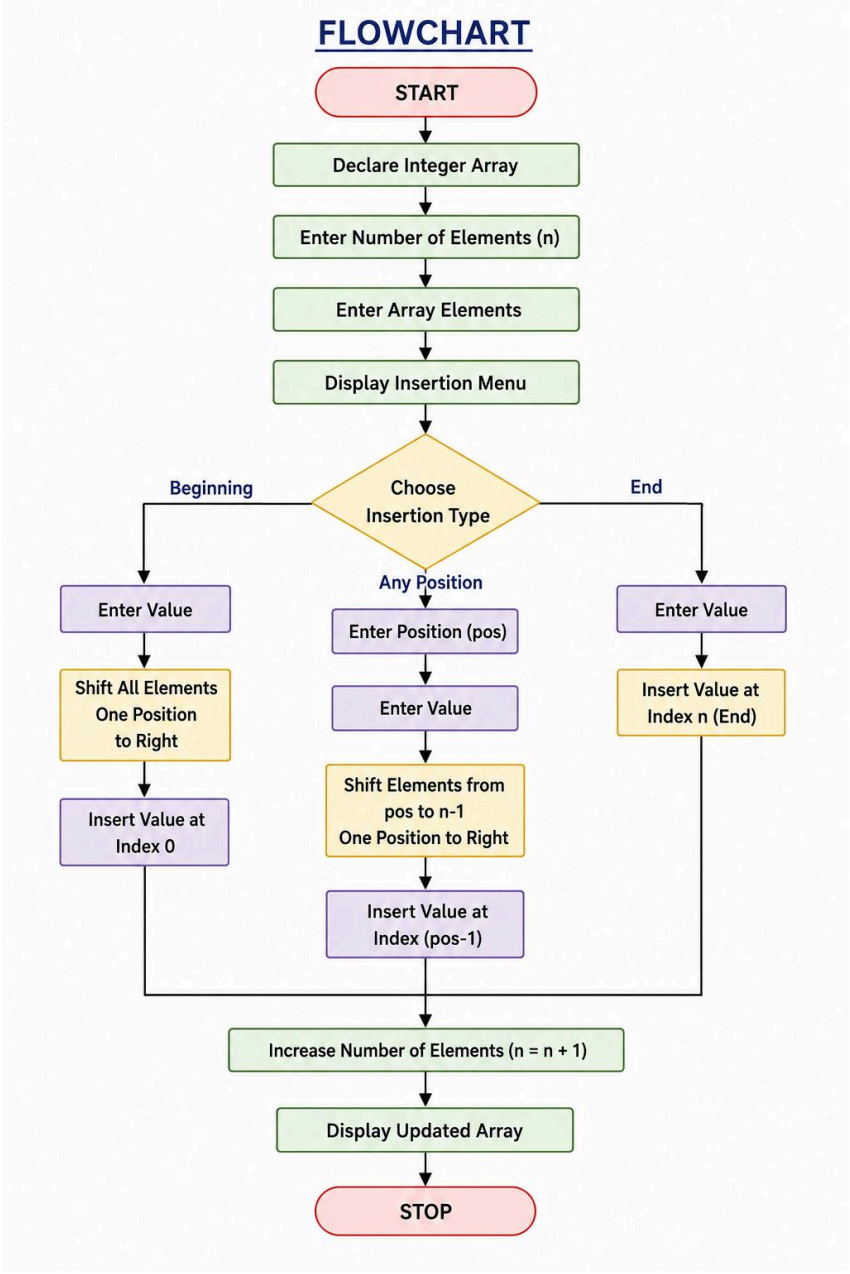
Updated Array:
10 20 30 40 50 60
```

Exp 4 · Array deletion · Output (1/3)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 1
Deleted element: 10
Updated Array:
20 30 40 50
```

Exp 3 · Array insertion · Flowchart (manual)



Exp 3 · Array insertion · Program (part 2/2)

```
        if (pos < 1 || pos > n + 1)
        {
            cout << "Invalid position.";
            return 0;
        }

        cout << "Enter Value: ";
        cin >> value;

        for (int i = n; i >= pos; i--)
            arr[i] = arr[i - 1];

        arr[pos - 1] = value;
        n++;
    }
    else if (choice == 3)
    {
        cout << "Enter Value: ";
        cin >> value;

        arr[n] = value;
        n++;
    }
    else
    {
        cout << "Invalid choice.";
        return 0;
    }

    cout << "\nUpdated Array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

Exp 4 · Array deletion · Output (2/3)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 2
Enter Position: 3
Deleted element: 30
Updated Array:
10 20 40 50
```

Exp 4 · Array deletion · Program (part 2/2)

```
        if (pos < 1 || pos > n)
        {
            cout << "Invalid position.";
            return 0;
        }

        cout << "Deleted element: " << arr[pos - 1];
        for (int i = pos - 1; i < n - 1; i++)
            arr[i] = arr[i + 1];
        n--;
    }
    else if (choice == 3)
    {
        cout << "Deleted element: " << arr[n - 1];
        n--;
    }
    else
    {
        cout << "Invalid choice.";
        return 0;
    }

    cout << "\nUpdated Array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

Exp 4 · Array deletion · Output (3/3)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 3
Deleted element: 50
Updated Array:
10 20 30 40
```

Exp 3 · Array insertion · Program (part 1/2)

```
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, value, pos, choice;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter array elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    if (n >= 100)
    {
        cout << "Array is full. Insertion not possible.";
        return 0;
    }

    cout << "\n1. Insert at Beginning";
    cout << "\n2. Insert at Any Position";
    cout << "\n3. Insert at End";
    cout << "\nEnter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
        cout << "Enter value: ";
        cin >> value;

        for (int i = n; i > 0; i--)
            arr[i] = arr[i - 1];

        arr[0] = value;
        n++;
    }
    else if (choice == 2)
    {
        cout << "Enter Position: ";
        cin >> pos;
```

Exp 4 · Array deletion · Program (part 1/2)

```
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, pos, choice;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter array elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    if (n <= 0)
    {
        cout << "Array is empty. Deletion not possible.";
        return 0;
    }

    cout << "\n1. Delete at Beginning";
    cout << "\n2. Delete at Any Position";
    cout << "\n3. Delete at End";
    cout << "\nEnter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
        cout << "Deleted element: " << arr[0];
        for (int i = 0; i < n - 1; i++)
            arr[i] = arr[i + 1];
        n--;
    }
    else if (choice == 2)
    {
        cout << "Enter Position: ";
        cin >> pos;
```

Exp 5 · Matrix multiplication · Program (part 1/2)

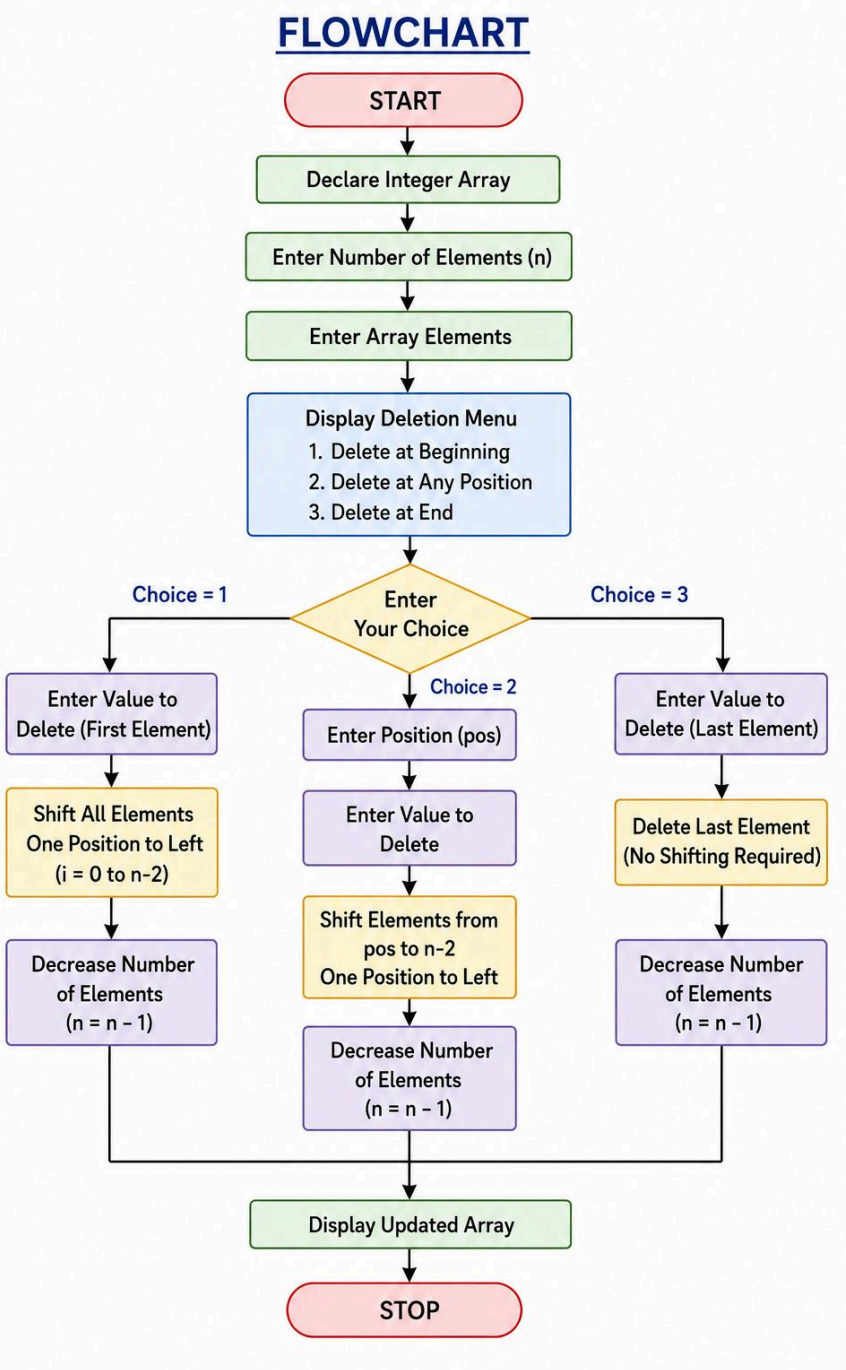
```
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            cin >> matrix[i][j];
}

// Function to multiply two matrices: C = A x B
void multiplyMatrix(int A[][10], int B[][10], int C[][10], int r1, int c1, int c2)
{
    for (int i = 0; i < r1; i++)
    {
        for (int j = 0; j < c2; j++)
        {
            C[i][j] = 0;
            for (int k = 0; k < c1; k++)
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
        }
    }
}

// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            cout << matrix[i][j] << " ";
        cout << endl;
    }
}
```

Exp 4 · Array deletion · Flowchart (manual)

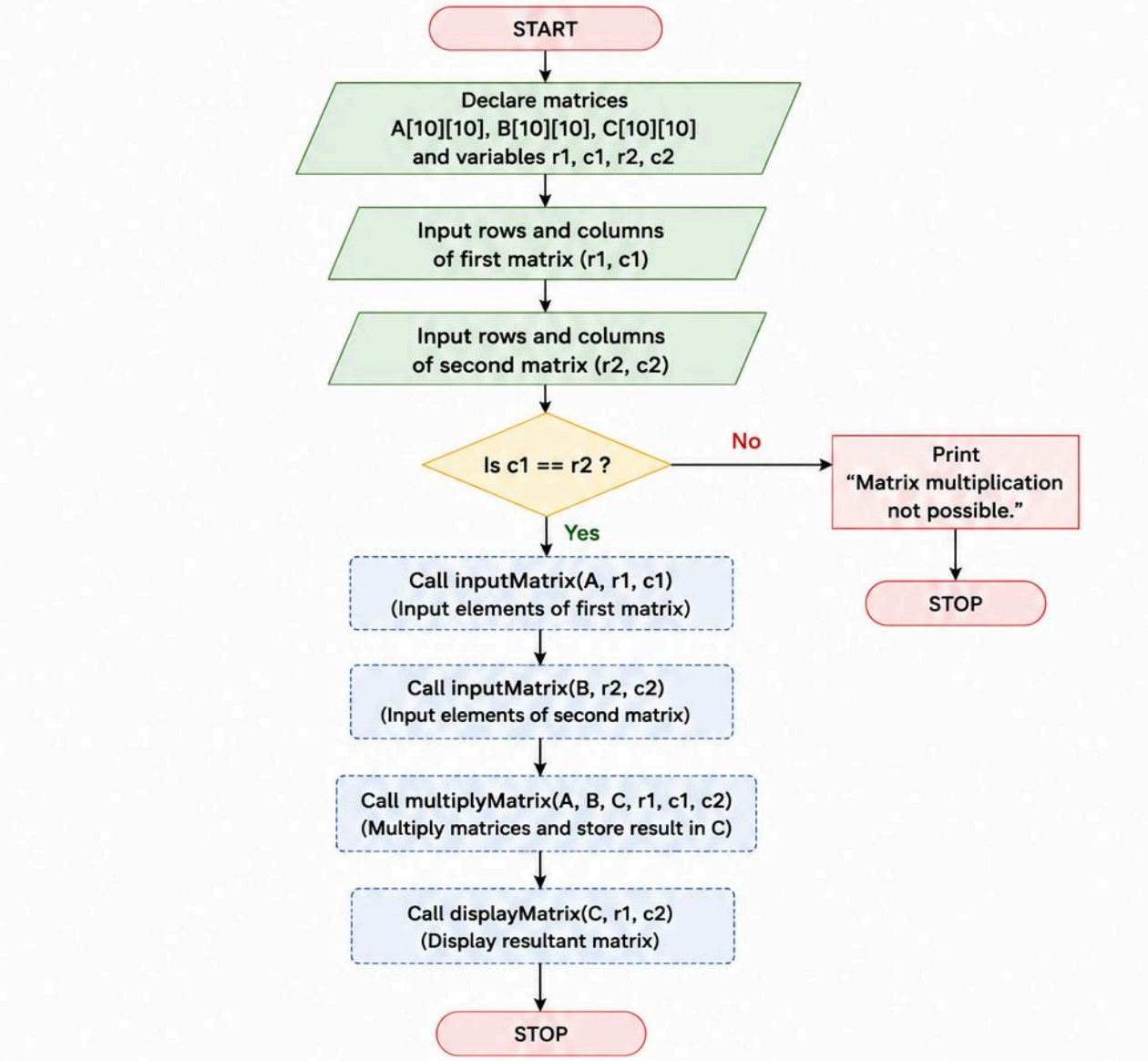


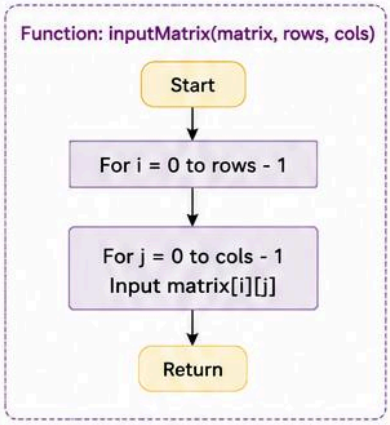
Exp 5 · Matrix multiplication · Output

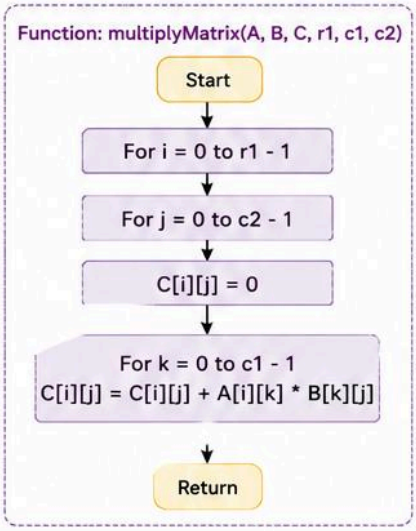
```
Enter rows and columns of first matrix: 2 2
Enter rows and columns of second matrix: 2 2
Enter elements of first matrix:
1 2
3 4
Enter elements of second matrix:
5 6
7 8

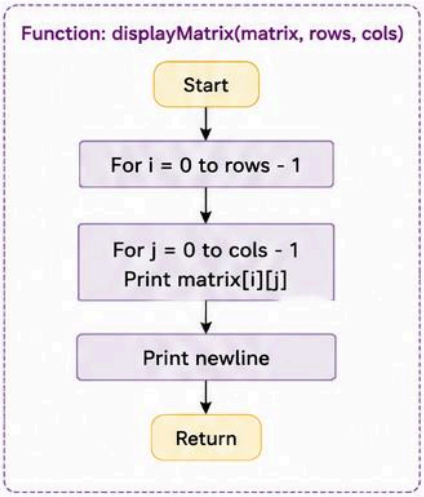
Resultant Matrix:
19 22
43 50
```

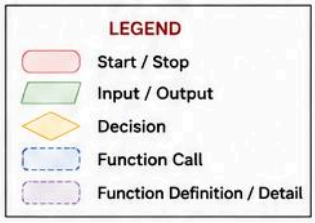
FLOWCHART FOR MATRIX MULTIPLICATION USING MODULAR PROGRAMMING











Exp 5 · Matrix multiplication · Program (part 2/2)

```
int main()
{
    int A[10][10], B[10][10], C[10][10];
    int r1, c1, r2, c2;

    cout << "Enter rows and columns of first matrix: ";
    cin >> r1 >> c1;

    cout << "Enter rows and columns of second matrix: ";
    cin >> r2 >> c2;

    if (r1 < 1 || c1 < 1 || r2 < 1 || c2 < 1 || r1 > 10 || c1 > 10 || r2 > 10 || c2 > 10)
    {
        cout << "Sizes must be between 1 and 10.";
        return 0;
    }

    // Check multiplication condition
    if (c1 != r2)
    {
        cout << "Matrix multiplication not possible.";
        return 0;
    }

    cout << "Enter elements of first matrix:\n";
    inputMatrix(A, r1, c1);

    cout << "Enter elements of second matrix:\n";
    inputMatrix(B, r2, c2);

    multiplyMatrix(A, B, C, r1, c1, c2);

    cout << "\nResultant Matrix:\n";
    displayMatrix(C, r1, c2);

    return 0;
}
```

Exp 6 · Matrix transpose · Program (part 1/2)

```
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            cin >> matrix[i][j];
}

// Function to find transpose of matrix
void transposeMatrix(int A[][10], int T[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            T[j][i] = A[i][j];
}

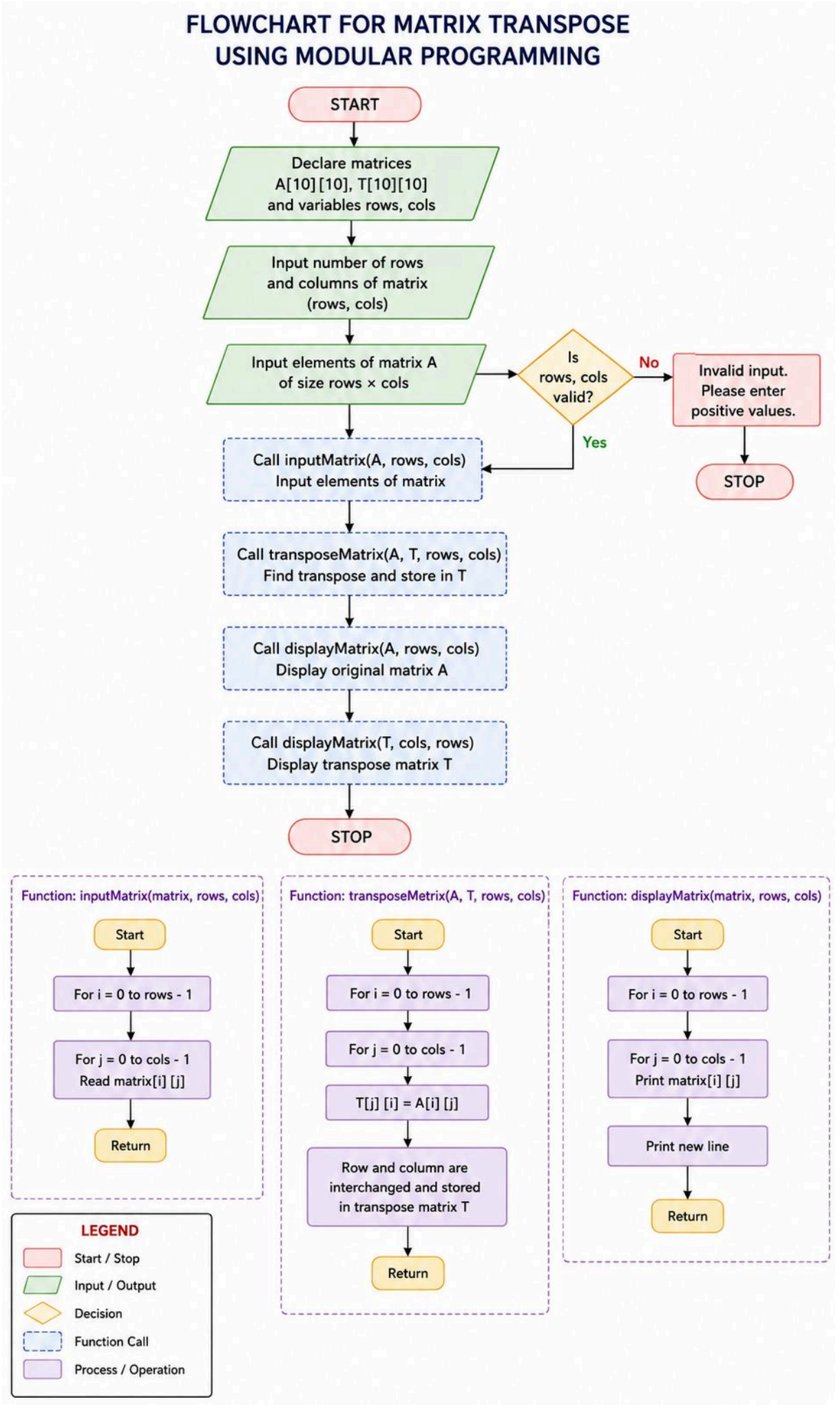
// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            cout << matrix[i][j] << " ";
        cout << endl;
    }
}
```

Exp 6 · Matrix transpose · Output

```
Enter number of rows and columns of matrix: 2 3
Enter elements of matrix:
1 2 3
4 5 6

Original Matrix:
1 2 3
4 5 6

Transpose Matrix:
1 4
2 5
3 6
```


FLOWCHART FOR MATRIX TRANSPOSE
USING MODULAR PROGRAMMING
START
Declare matrices
A[10][10], T[10][10]
and variables rows, cols
Input number of rows
and columns of matrix
(rows, cols)
Input elements of matrix A
of size rows × cols
Is
rows, cols
valid?
No
Invalid input.
Please enter
positive values.
STOP
Yes
Call inputMatrix(A, rows, cols)
Input elements of matrix
Call transposeMatrix(A, T, rows, cols)
Find transpose and store in T
Call displayMatrix(A, rows, cols)
Display original matrix A
Call displayMatrix(T, cols, rows)
Display transpose matrix T
STOP
Function: inputMatrix(matrix, rows, cols)
Start
For i = 0 to rows - 1
For j = 0 to cols - 1
Read matrix[i] [j]
Return
Function: transposeMetrix(A, T, rows, cols)
Start
For i = 0 to rows - 1
For j = 0 to cols - 1
T[j] [i] = A[i] [j]
Row and column are
interchanged and stored
in transpose matrix T
Return
Function: displayMatrix(matrix, rows, cols)
Start
For i = 0 to rows - 1
For j = 0 to cols - 1
Print matrix[i] [j]
Print new line
Return
LEGEND
Start / Stop
Input / Output
Decision
Function Call
Process / Operation

Exp 6 · Matrix transpose · Program (part 2/2)

```
int main()
{
    int A[10][10], T[10][10];
    int rows, cols;

    cout << "Enter number of rows and columns of matrix: ";
    cin >> rows >> cols;

    if (rows < 1 || cols < 1 || rows > 10 || cols > 10)
    {
        cout << "Sizes must be between 1 and 10.";
        return 0;
    }

    cout << "Enter elements of matrix:\n";
    inputMatrix(A, rows, cols);

    transposeMatrix(A, T, rows, cols);

    cout << "\nOriginal Matrix:\n";
    displayMatrix(A, rows, cols);

    cout << "\nTranspose Matrix:\n";
    displayMatrix(T, cols, rows);

    return 0;
}
```

Exp 7 · Insertion sort · Output

```
Enter number of elements: 6
Enter elements of the array:
5 2 4 6 1 3
Original Array: 5 2 4 6 1 3
Sorted Array: 1 2 3 4 5 6
```

Exp 7 · Insertion sort · Program (part 1/2)

```
#include <iostream>
using namespace std;

// Function to perform Insertion Sort
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;

        // Shift elements greater than key one position to the right
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

// Function to display the array
void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```

Exp 8 · Quick sort + comparison · Program (part 1/4)

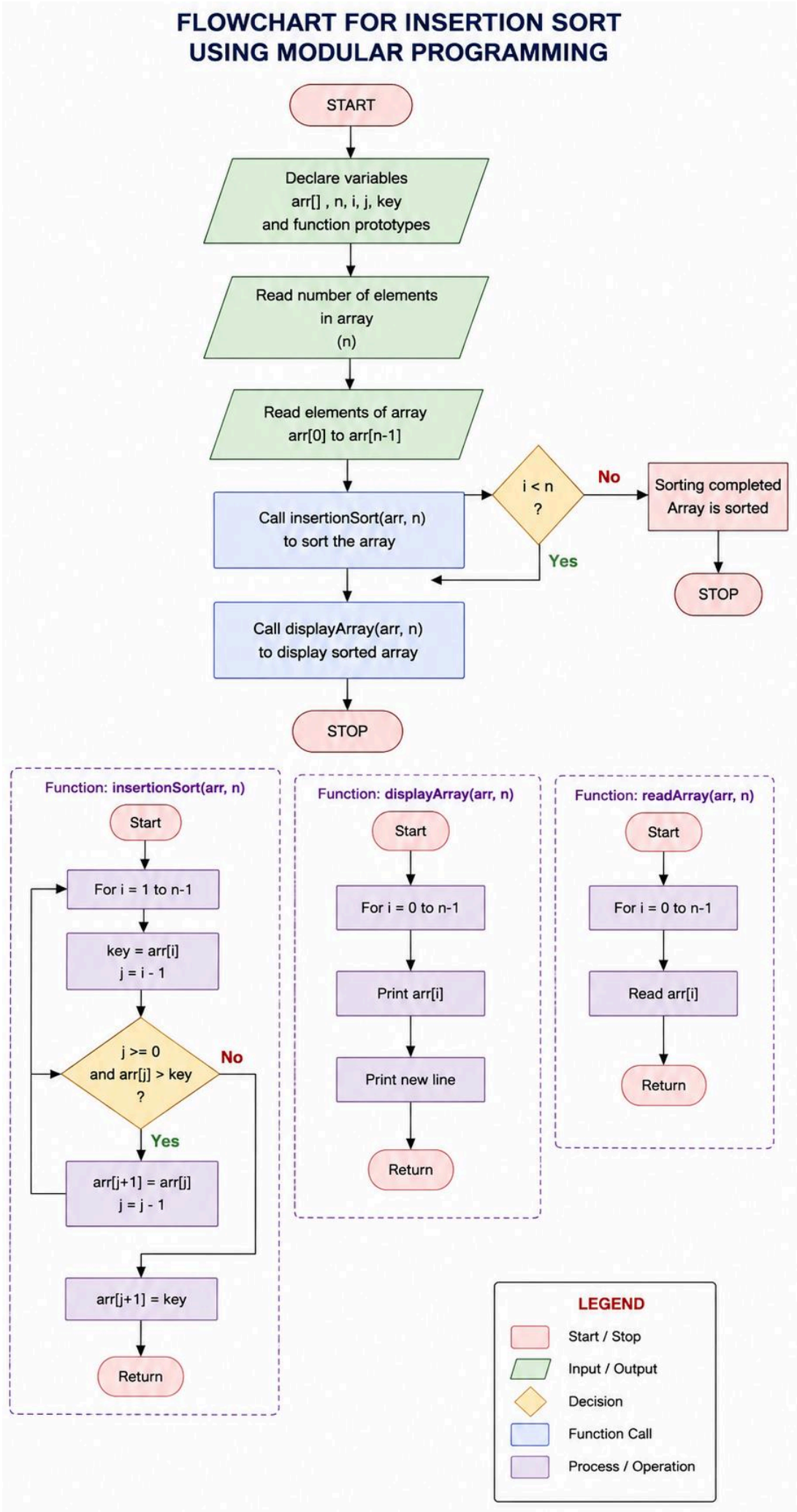
```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

const int BIG = 10000;

void swapElements(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

int partitionArray(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionArray(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```


FLOWCHART FOR INSERTION SORT
USING MODULAR PROGRAMMING
START
Declare variables
arr[] , n, i, j, key
and function prototypes
Read number of elements
in array
(n)
Read elements of array
arr[0] to arr[n-1]
Call insertionSort(arr, n)
to sort the array
i < n
?
No
Yes
Sorting completed
Array is sorted
STOP
Call displayArray(arr, n)
to display sorted array
STOP
Function: insertionSort(arr, n)
Start
For i = 1 to n-1
key = arr[i]
j = i - 1
j >= 0
and arr[j] > key
?
No
Yes
arr[j+1] = arr[j]
j = j - 1
arr[j+1] = key
Return
Function: displayArray(arr, n)
Start
For i = 0 to n-1
Print arr[i]
Print new line
Return
Function: readArray(arr, n)
Start
For i = 0 to n-1
Read arr[i]
Return
LEGEND
Start / Stop
Input / Output
Decision
Function Call
Process / Operation

Exp 7 · Insertion sort · Program (part 2/2)

```
int main()
{
    int n;
    int arr[100];

    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Original Array: ";
    displayArray(arr, n);

    insertionSort(arr, n);

    cout << "Sorted Array: ";
    displayArray(arr, n);

    return 0;
}
```

Exp 8 · Quick sort + comparison · Output

```
Enter number of elements: 6
Enter elements of the array:
10 7 8 9 1 5
Quick Sort     : 1 5 7 8 9 10
Bubble Sort    : 1 5 7 8 9 10
Selection Sort : 1 5 7 8 9 10
Insertion Sort : 1 5 7 8 9 10

Time to sort 10000 random numbers:
Quick Sort     : 1 ms
Bubble Sort    : 114 ms
Selection Sort : 53 ms
Insertion Sort : 37 ms
```

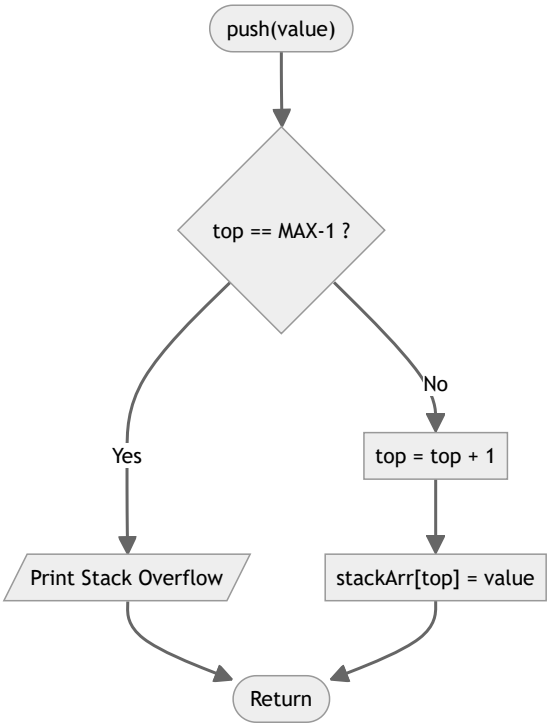
Exp 8 · Quick sort + comparison · Program (part 2/4)

```
// Bubble Sort: neighbours swap until the largest "bubbles" to the end
void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++)
            if (arr[j] > arr[j + 1])
            {
                swapElements(arr[j], arr[j + 1]);
                swapped = true;
            }
        if (!swapped)
            break;
    }
}

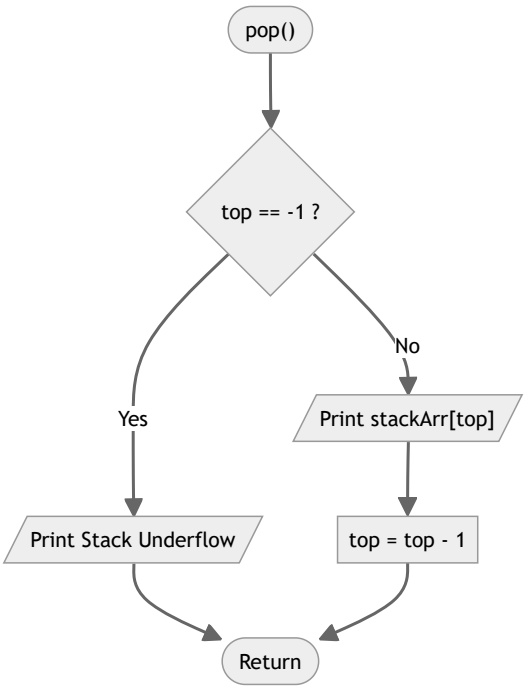
// Selection Sort: pick the smallest of the rest, put it at position i
void selectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIndex])
                minIndex = j;
        swapElements(arr[i], arr[minIndex]);
    }
}

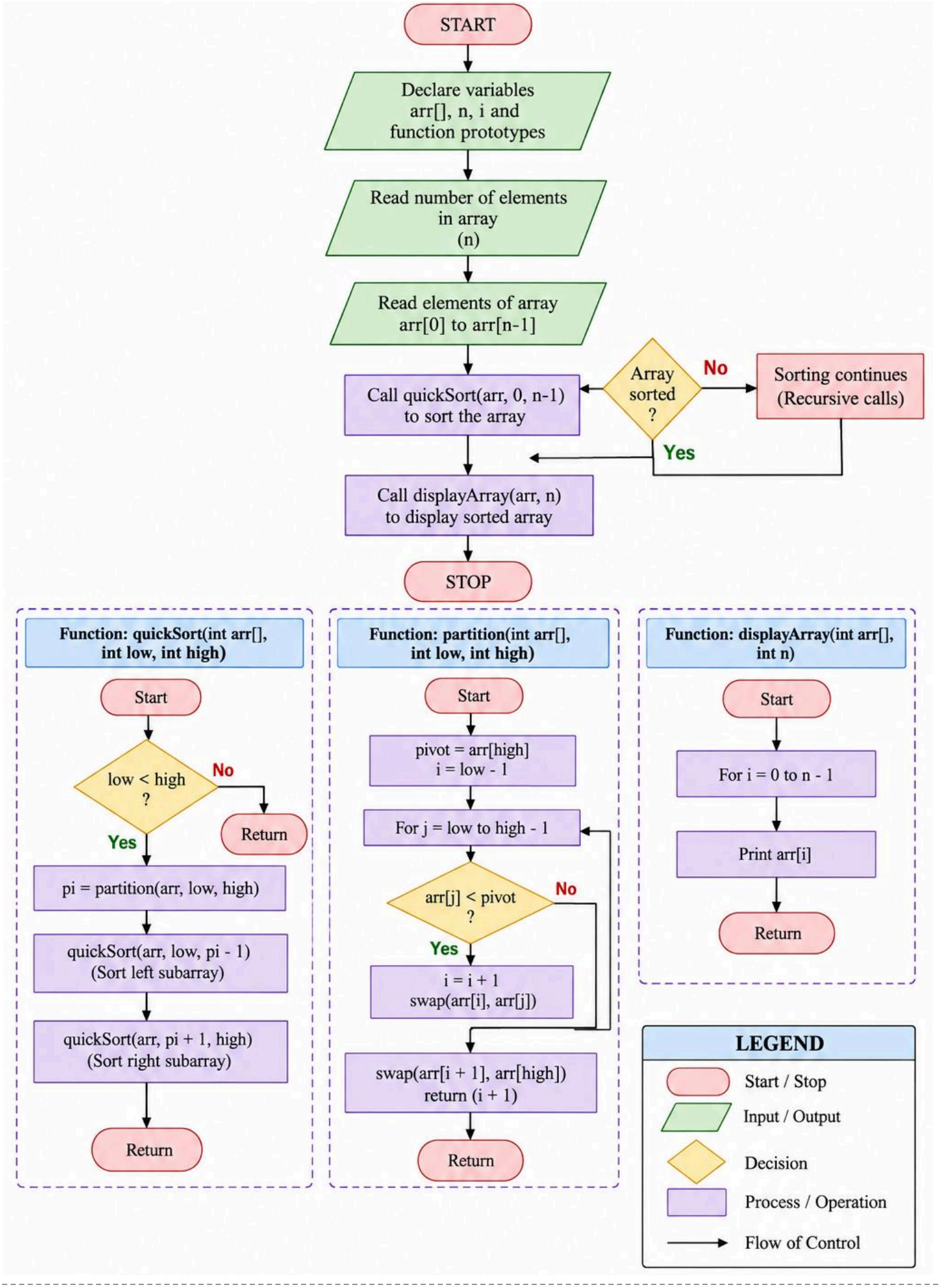
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Exp 9 · Stack (probable) · Flowchart (2)



Exp 9 · Stack (probable) · Flowchart (3)




START
Declare variables arr[], n, i and function prototypes
Read number of elements in array (n)
Read elements of array arr[0] to arr[n-1]
Call quickSort(arr, 0, n-1) to sort the array
Array sorted ?
No
Sorting continues (Recursive calls)
Yes
Call displayArray(arr, n) to display sorted array
STOP
Function: quickSort(int arr[], int low, int high)
Start
low < high ?
No
Return
Yes
pi = partition(arr, low, high)
quickSort(arr, low, pi - 1) (Sort left subarray)
quickSort(arr, pi + 1, high) (Sort right subarray)
Return
Function: partition(int arr[], int low, int high)
Start
pivot = arr[high]
i = low - 1
For j = low to high - 1
arr[j] < pivot ?
No
Yes
i = i + 1
swap(arr[i], arr[j])
swap(arr[i + 1], arr[high])
return (i + 1)
Return
Function: displayArray(int arr[], int n)
Start
For i = 0 to n - 1
Print arr[i]
Return
LEGEND
Start / Stop
Input / Output
Decision
Process / Operation
Flow of Control

Exp 8 · Quick sort + comparison · Program (part 3/4)

```
void copyArray(int source[], int dest[], int n)
{
    for (int i = 0; i < n; i++)
        dest[i] = source[i];
}

void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int original[BIG], work[BIG];

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> original[i];
```

Exp 10 · Queue · Program (part 1/3)

```
#include <iostream>
using namespace std;

#define MAX 5
int queueArr[MAX];
int front = -1;
int rear = -1;

void enqueue(int value)
{
    if (rear == MAX - 1)
    {
        cout << "Queue Overflow!" << endl;
    }
    else
    {
        if (front == -1)
            front = 0;
        rear++;
        queueArr[rear] = value;
        cout << value << " inserted into queue." << endl;
    }
}
```

Exp 8 · Quick sort + comparison · Program (part 4/4)

```
    // Part 1: every algorithm sorts the SAME input (each gets its own copy)
    copyArray(original, work, n);  quickSort(work, 0, n - 1);
    cout << "Quick Sort     : ";  displayArray(work, n);
    copyArray(original, work, n);  bubbleSort(work, n);
    cout << "Bubble Sort    : ";  displayArray(work, n);
    copyArray(original, work, n);  selectionSort(work, n);
    cout << "Selection Sort : ";  displayArray(work, n);
    copyArray(original, work, n);  insertionSort(work, n);
    cout << "Insertion Sort : ";  displayArray(work, n);

    // Part 2: timing needs a big array - 6 numbers sort in ~0 ms
    srand(1);
    for (int i = 0; i < BIG; i++)
        original[i] = rand() % 100000;

    cout << "\nTime to sort " << BIG << " random numbers:" << endl;
    clock_t start, end;

    copyArray(original, work, BIG);
    start = clock();  quickSort(work, 0, BIG - 1);  end = clock();
    cout << "Quick Sort     : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

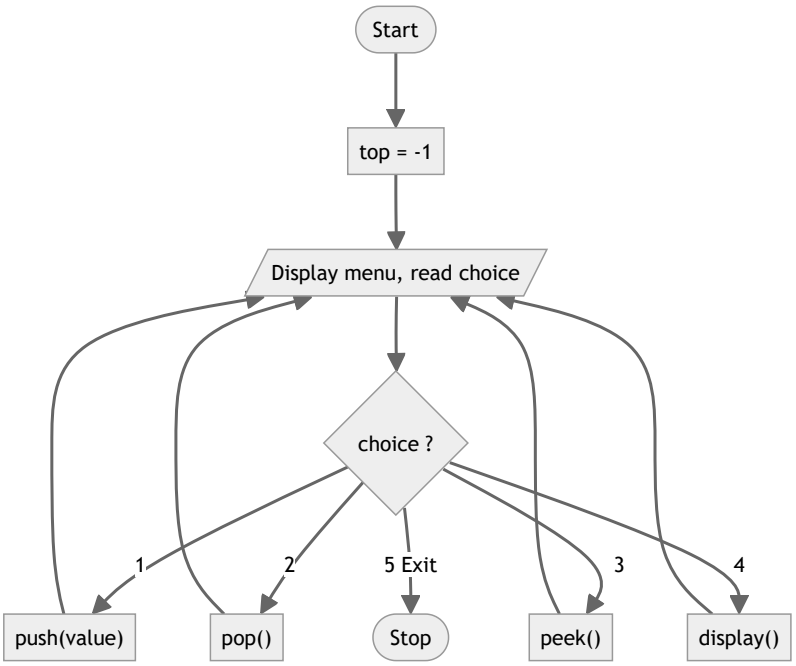
    copyArray(original, work, BIG);
    start = clock();  bubbleSort(work, BIG);  end = clock();
    cout << "Bubble Sort    : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  selectionSort(work, BIG);  end = clock();
    cout << "Selection Sort : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  insertionSort(work, BIG);  end = clock();
    cout << "Insertion Sort : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    return 0;
}
```

Exp 9 · Stack (probable) · Flowchart (1)



Exp 9 · Stack (probable) · Program (part 1/2)

```
#include <iostream>
using namespace std;

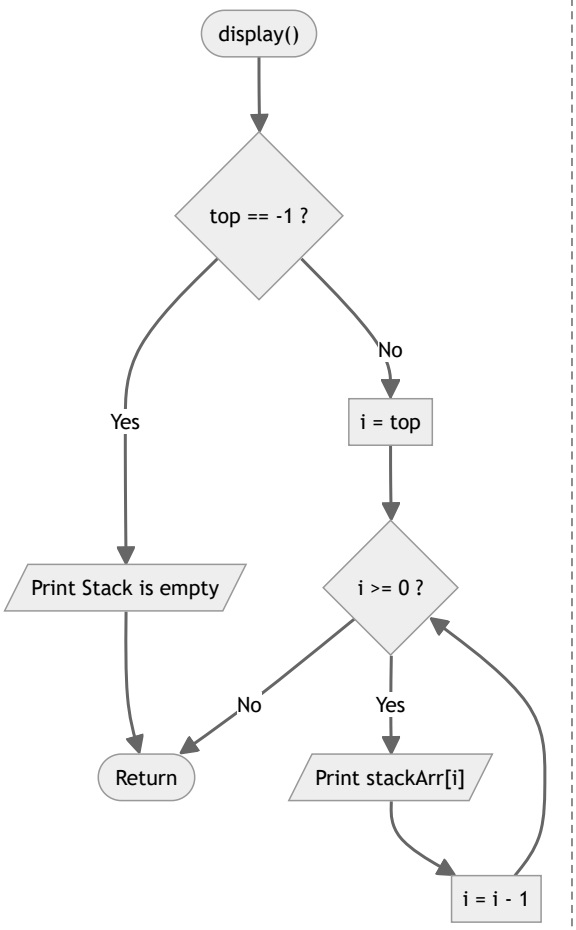
#define MAX 5
int stackArr[MAX];
int top = -1;

void push(int value)
{
    if (top == MAX - 1)
        cout << "Stack Overflow!" << endl;
    else
    {
        top++;
        stackArr[top] = value;
        cout << value << " pushed into stack." << endl;
    }
}

void pop()
{
    if (top == -1)
        cout << "Stack Underflow!" << endl;
    else
    {
        cout << stackArr[top] << " popped from stack." << endl;
        top--;
    }
}

void peek()
{
    if (top == -1)
        cout << "Stack is empty." << endl;
    else
        cout << "Top element is " << stackArr[top] << endl;
}
```

Exp 9 · Stack (probable) · Flowchart (4)



Exp 9 · Stack (probable) · Output

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 10
10 pushed into stack.
Enter your choice: 1
Enter value to push: 20
20 pushed into stack.
Enter your choice: 1
Enter value to push: 30
30 pushed into stack.
Enter your choice: 4
Stack elements (top to bottom):
30 20 10
Enter your choice: 3
Top element is 30
Enter your choice: 2
30 popped from stack.
Enter your choice: 4
Stack elements (top to bottom):
20 10
Enter your choice: 2
20 popped from stack.
Enter your choice: 2
10 popped from stack.
Enter your choice: 2
Stack Underflow!
Enter your choice: 5
Program terminated.
```

Exp 9 · Stack (probable) · Program (part 2/2)

```
void display()
{
    if (top == -1)
        cout << "Stack is empty." << endl;
    else
    {
        cout << "Stack elements (top to bottom):" << endl;
        for (int i = top; i >= 0; i--)
            cout << stackArr[i] << " ";
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Push" << endl;
        cout << "2. Pop" << endl;
        cout << "3. Peek" << endl;
        cout << "4. Display" << endl;
        cout << "5. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to push: ";
            cin >> value;
            push(value);
        }
        else if (choice == 2)
            pop();
        else if (choice == 3)
            peek();
        else if (choice == 4)
            display();
        else if (choice == 5)
            cout << "Program terminated." << endl;
        else
            cout << "Invalid choice!" << endl;
    } while (choice != 5);

    return 0;
}
```

Exp 11 · Singly linked list · Program (part 1/3)

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = NULL;

// Insert at the end
void insert(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        Node *temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
    cout << value << " inserted into list." << endl;
}
```

Exp 10 · Queue · Program (part 3/3)

```
    do
    {
        cout << "\n1. Enqueue" << endl;
        cout << "2. Dequeue" << endl;
        cout << "3. Display" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to enqueue: ";
            cin >> value;
            enqueue(value);
        }
        else if (choice == 2)
        {
            dequeue();
        }
        else if (choice == 3)
        {
            display();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

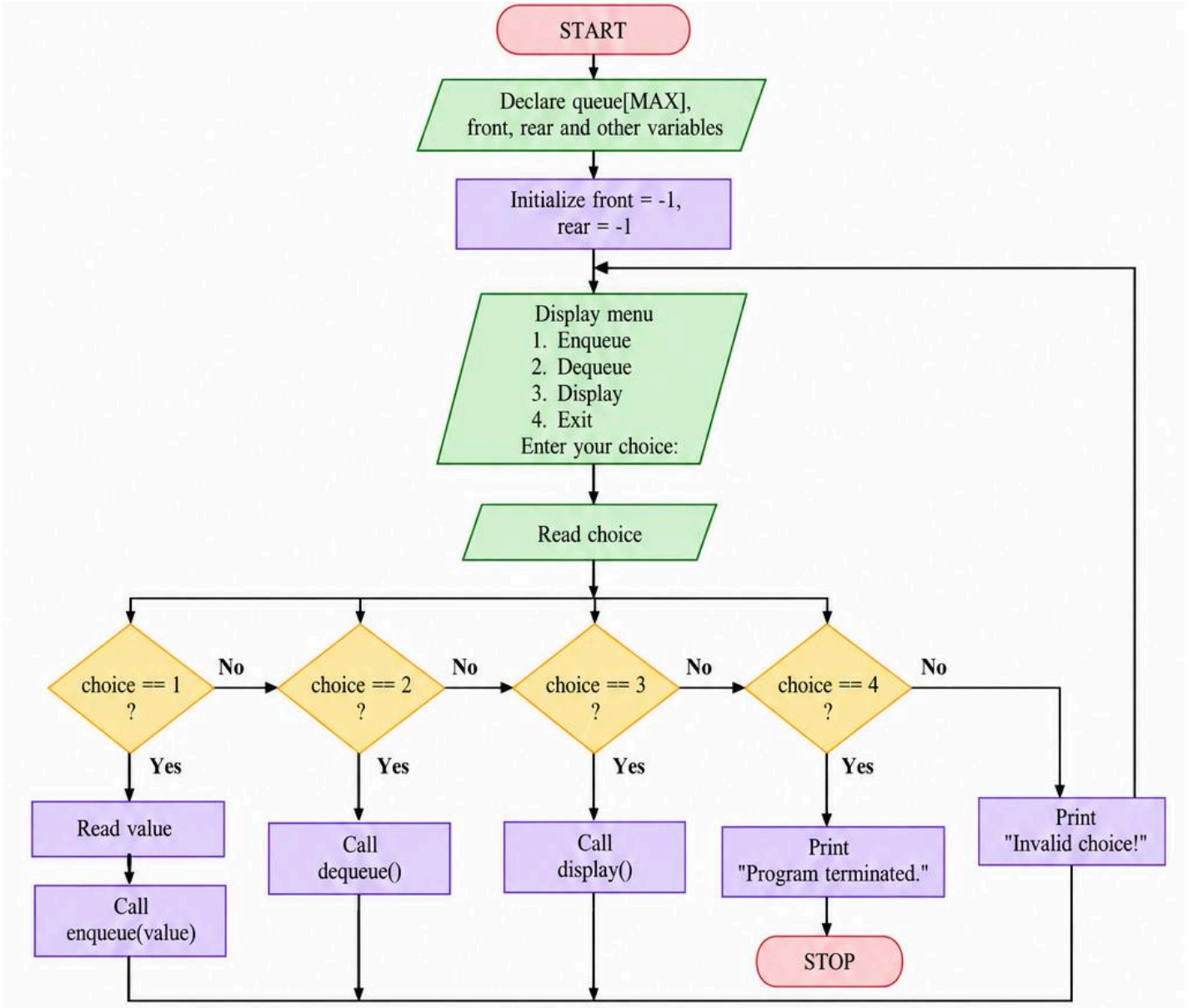
    return 0;
}
```

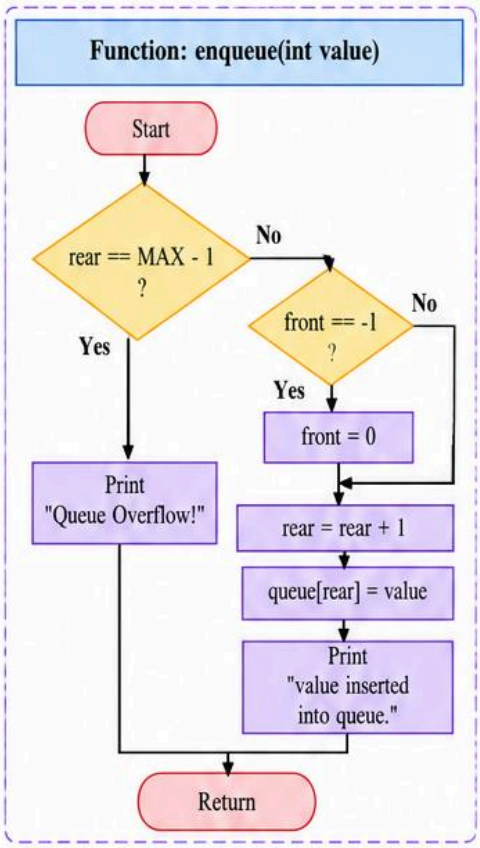
Exp 10 · Queue · Output

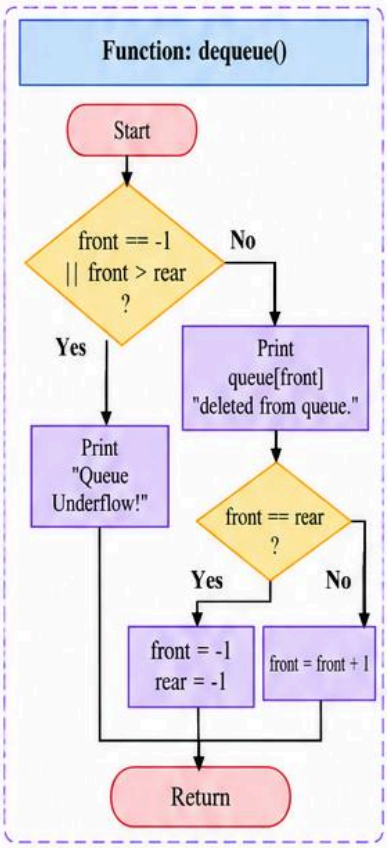
```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10
10 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 20
20 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 30
30 inserted into queue.
Enter your choice: 3
Queue elements are:
10 20 30
Enter your choice: 2
10 deleted from queue.
Enter your choice: 3
Queue elements are:
20 30
Enter your choice: 4
Program terminated.
```

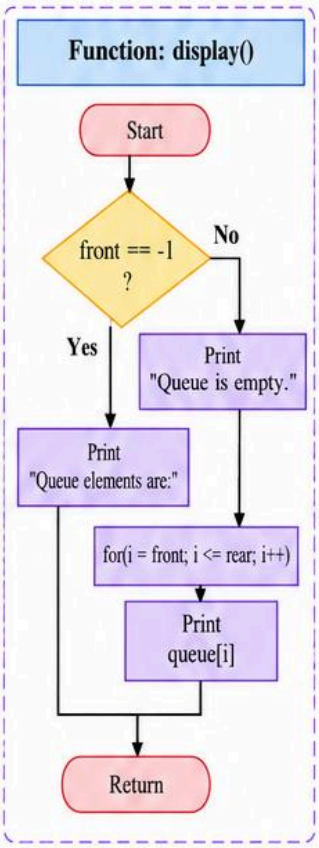
Exp 11 · Singly linked list · Output

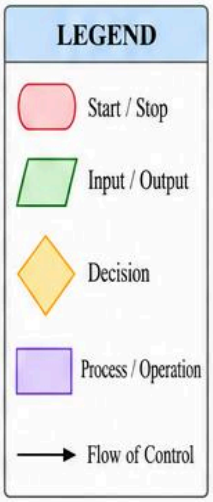
```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 10
10 inserted into list.
Enter your choice: 1
Enter value to insert: 20
20 inserted into list.
Enter your choice: 1
Enter value to insert: 30
30 inserted into list.
Enter your choice: 3
Linked List: 10 20 30
Enter your choice: 2
10 deleted from list.
Enter your choice: 3
Linked List: 20 30
Enter your choice: 4
Program terminated.
```


START
Declare queue[MAX], front, rear and other variables
Initialize front = -1, rear = -1
Display menu
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice:
Read choice
choice == 1 ?
No
choice == 2 ?
No
choice == 3 ?
No
choice == 4 ?
No
Yes
Read value
Call enqueue(value)
Call dequeue()
Call display()
Print "Program terminated."
STOP
Print "Invalid choice!"


Function: enqueue(int value)
Start
rear == MAX - 1 ?
No
Yes
front == -1 ?
No
Yes
front = 0
Print "Queue Overflow!"
rear = rear + 1
queue[rear] = value
Print "value inserted into queue."
Return


Function: dequeue()
Start
front == -1 || front > rear ?
No
Yes
Print "Queue Underflow!"
Print queue[front] "deleted from queue."
front == rear ?
Yes
No
front = -1 rear = -1
front = front + 1
Return


Function: display()
Start
front == -1 ?
No
Yes
Print "Queue is empty."
Print "Queue elements are:"
for(i = front; i <= rear; i++)
Print queue[i]
Return


LEGEND
Start / Stop
Input / Output
Decision
Process / Operation
Flow of Control

Exp 10 · Queue · Program (part 2/3)

```
void dequeue()
{
    if (front == -1)
    {
        cout << "Queue Underflow!" << endl;
    }
    else
    {
        cout << queueArr[front] << " deleted from queue." << endl;
        if (front == rear)
        {
            front = -1;
            rear = -1;
        }
        else
        {
            front++;
        }
    }
}

void display()
{
    if (front == -1)
    {
        cout << "Queue is empty." << endl;
    }
    else
    {
        cout << "Queue elements are:" << endl;
        for (int i = front; i <= rear; i++)
            cout << queueArr[i] << " ";
        cout << endl;
    }
}

int main()
{
    int choice, value;
```

Exp 11 · Singly linked list · Program (part 3/3)

```
    do
    {
        cout << "\n1. Insert" << endl;
        cout << "2. Delete" << endl;
        cout << "3. Traverse" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to insert: ";
            cin >> value;
            insert(value);
        }
        else if (choice == 2)
        {
            deleteNode();
        }
        else if (choice == 3)
        {
            traverse();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```

Exp 11 · Singly linked list · Program (part 2/3)

```
// Delete from the beginning
void deleteNode()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << temp->data << " deleted from list." << endl;
        head = head->next;
        delete temp;
    }
}

void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << "Linked List: ";
        while (temp != NULL)
        {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }
}

int main()
{
    int choice, value;
```

Exp 12 · Circular linked list · Program (part 1/3)

```
#include <iostream>
using namespace std;

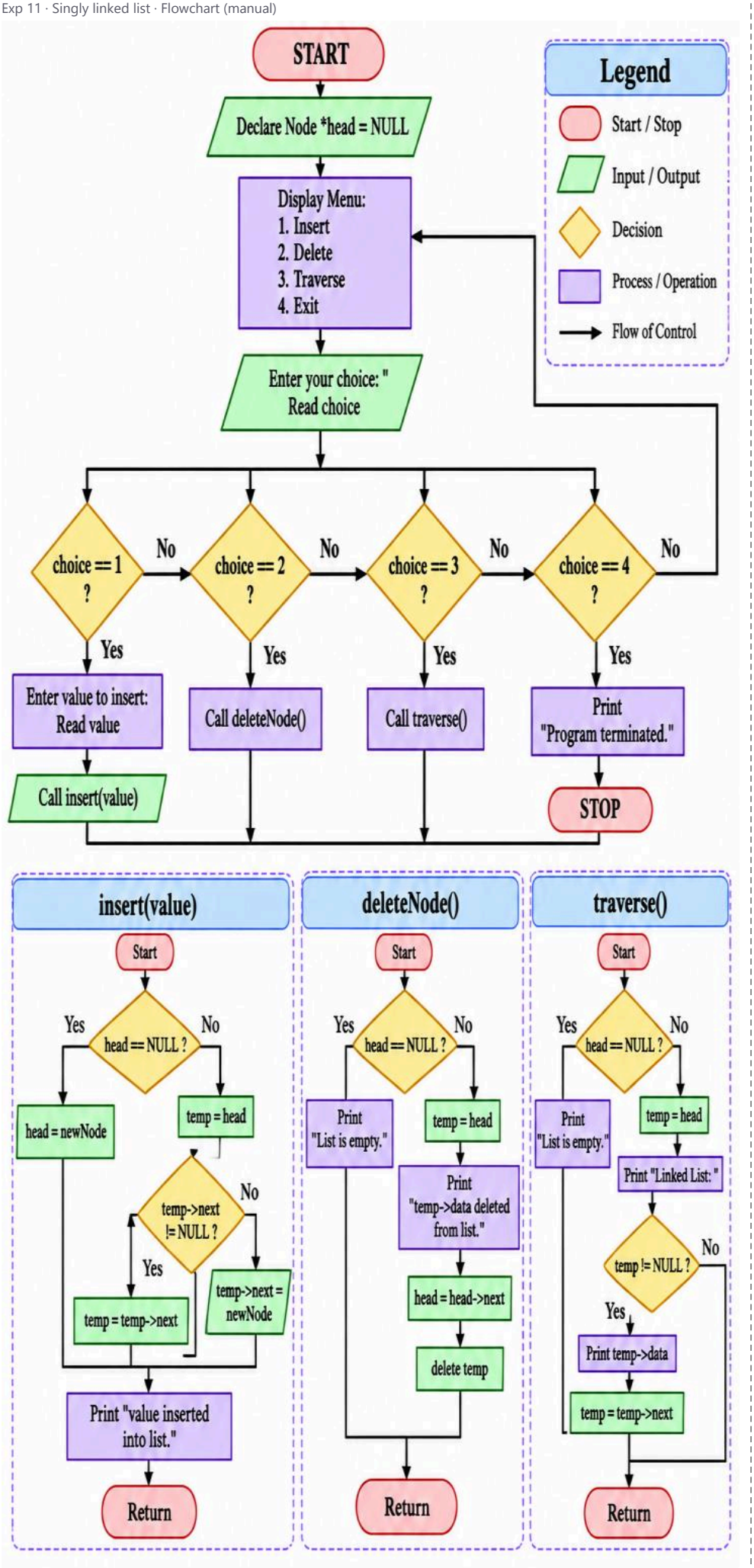
struct Node
{
    int data;
    Node *next;
};

Node *head = NULL;

// Insert at the end
void insert(int value)
{
    Node *newNode = new Node;
    newNode->data = value;

    if (head == NULL)
    {
        head = newNode;
        newNode->next = head;
    }
    else
    {
        Node *temp = head;
        while (temp->next != head)
            temp = temp->next;
        temp->next = newNode;
        newNode->next = head;
    }
    cout << value << " inserted into list." << endl;
}
```

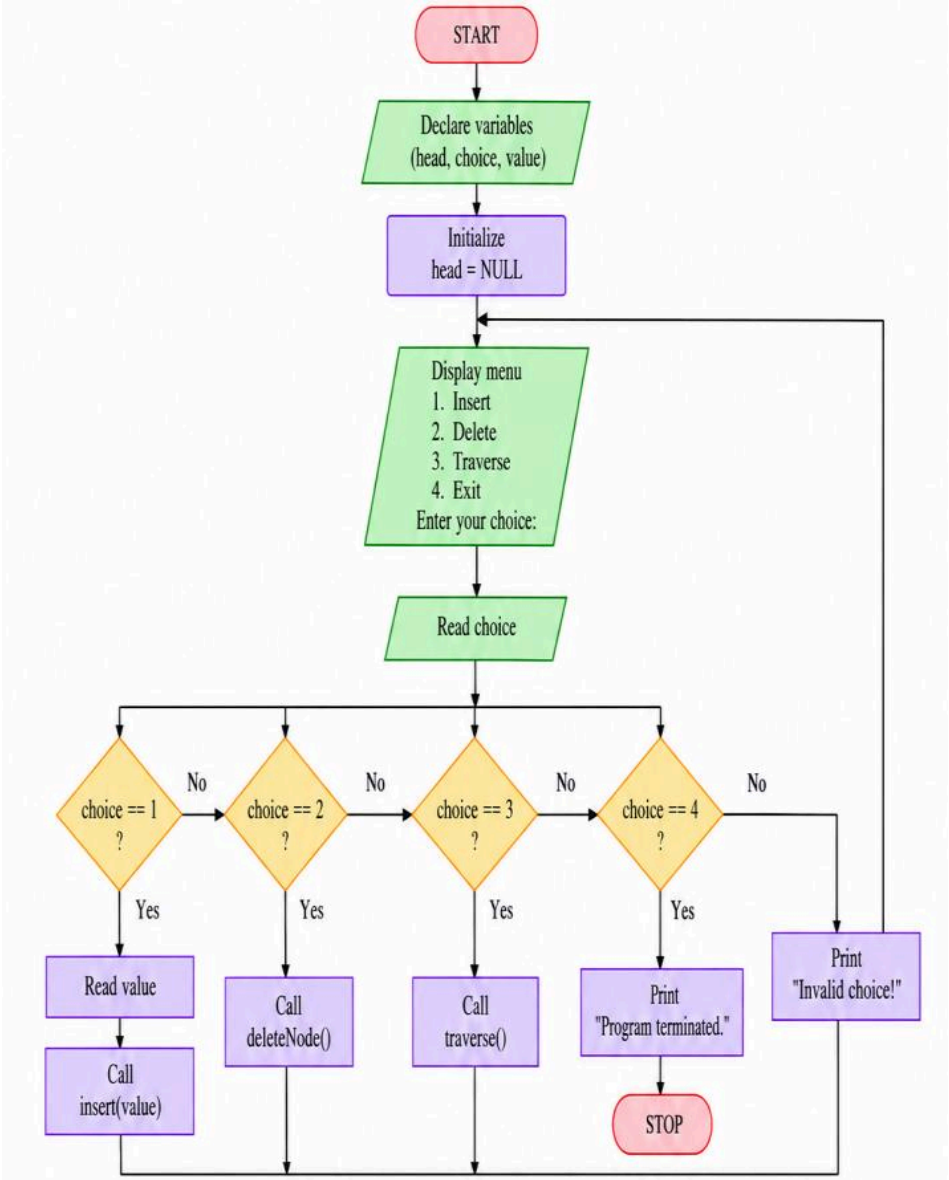
Exp 11 · Singly linked list · Flowchart (manual)

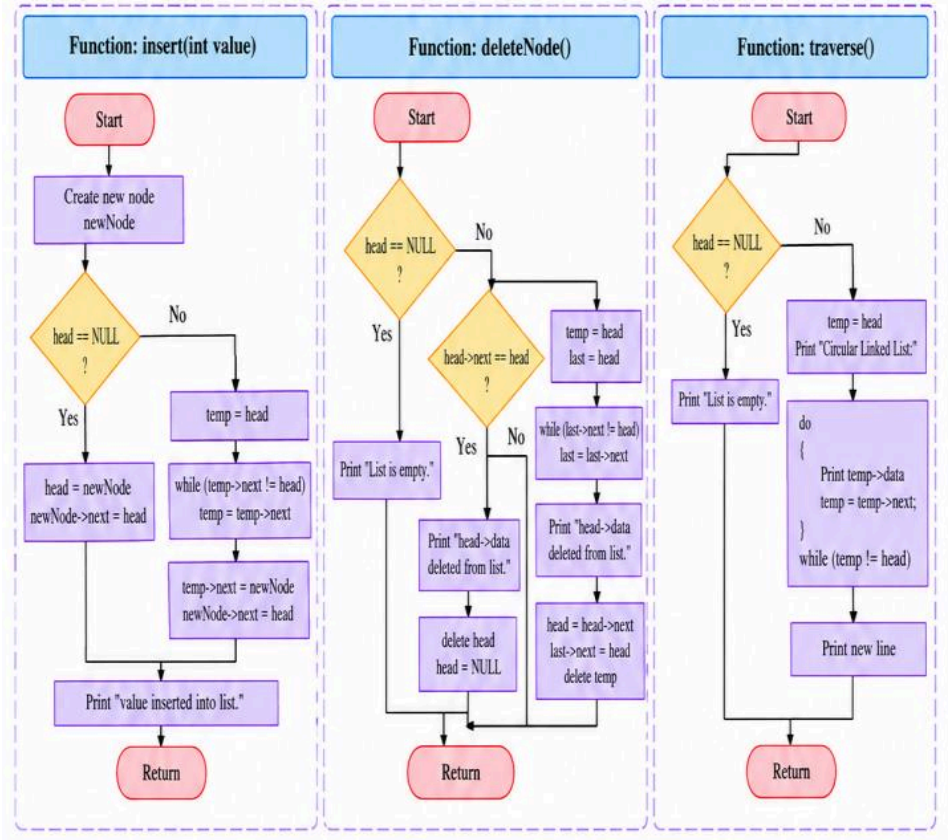


Exp 12 · Circular linked list · Output

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 10
10 inserted into list.
Enter your choice: 1
Enter value to insert: 20
20 inserted into list.
Enter your choice: 1
Enter value to insert: 30
30 inserted into list.
Enter your choice: 3
Circular Linked List: 10 20 30
Enter your choice: 2
10 deleted from list.
Enter your choice: 3
Circular Linked List: 20 30
Enter your choice: 4
Program terminated.
```

Exp 12 · Circular linked list · Flowchart (manual)





Exp 12 · Circular linked list · Program (part 2/3)

```
// Delete from the beginning
void deleteNode()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else if (head->next == head)
    {
        cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
    }
    else
    {
        Node *temp = head;
        Node *last = head;
        while (last->next != head)
            last = last->next;

        cout << head->data << " deleted from list." << endl;
        head = head->next;
        last->next = head;
        delete temp;
    }
}

void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << "Circular Linked List: ";
        do
        {
            cout << temp->data << " ";
            temp = temp->next;
        } while (temp != head);
        cout << endl;
    }
}
```

Exp 12 · Circular linked list · Program (part 3/3)

```
int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Insert" << endl;
        cout << "2. Delete" << endl;
        cout << "3. Traverse" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to insert: ";
            cin >> value;
            insert(value);
        }
        else if (choice == 2)
        {
            deleteNode();
        }
        else if (choice == 3)
        {
            traverse();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```